

Input forms
Advanced Design Scripting & Parametrics
Class 11

See reference
<https://developer.rhino3d.com/guides/eto/>

Rhino - Eto Guides

developer.rhino3d.com/guides/eto/



Eto Guides

Guides that apply to developing UIs across platforms using Eto.

 **Follow Along**

If you're new to Eto, or even an advanced user, it is recommended to open the [script editor](#) and copy/paste (or better yet write out!) the sample code so you can walk through and understand how Eto works.

Each page with samples will have both C# and Python 3 (The samples ARE NOT designed for Python 2), will work on both Windows and Mac and assumes little familiarity with Rhino.

If you're new to programming, this guide will not cover the basics, but will help you dive head-first into C# and/or Python.

Overview

- [What is Eto?](#)
- [Why Eto?](#)

Getting Started

- [Required Tools](#)

Fundamentals

Overview

- [Forms & Dialogs](#)
- [Existing Dialogs](#)
- [Containers](#)
 - [Panels](#)
 - [Tables](#)
 - [Stack Layouts](#)
 - [Dynamic Layouts](#)

Eto Native Controls

- [Basic Controls](#)
 - [Button](#)
 - [DropDown](#)
 - [Radio Buttons](#)
 - [Check Boxes](#)
 - [Text Box](#)
 - [Text Area](#)

Eto namespace reference

https://pages.picoe.ca/docs/api/html/R_Project_EtoForms.htm#!

The screenshot shows a web browser window with the address bar displaying `pages.picoe.ca/docs/api/html/R_Project_EtoForms.htm#!`. The page title is "Eto.Forms documentation". On the left, a sidebar menu shows the "Namespaces" section expanded, listing various classes and interfaces under the "Eto.Forms" namespace. The main content area is titled "Namespaces" and contains the text "Eto.Forms Cross Platform UI Framework". Below this, there is a table with two columns: "Namespace" and "Description".

Namespace	Description
Eto	Global platform, bindings, and helpers.
Eto.Drawing	Graphics, images, and drawing.
Eto.Forms	Application user interface and controls.
Eto.Forms.ThemedControls	Themed control handlers
Eto.IO	File Input/Output
Eto.Threading	Threading

Rhino – Eto tutorial

<https://developer.rhino3d.com/guides/rhinopython/eto-controls-python/>

Rhino - Eto Controls in Python

developer.rhino3d.com/guides/rhinopython/eto-controls-python/

RhinoDeveloper®

GuidesSamplesAPIVideosCommunity

search

Eto Controls in Python

- Buttons
- Calendar
- CheckBox
- ColorPicker
- ComboBox
- DateTimePicker
- DropDown
- GridView
- GroupBox
- ImageView
- Label
- LinkButton
- ListBox
- NumericUpDown
- PasswordBox
- ProgressBar
- RadioButtonList
- RichTextArea
- Slider
- Spinner
- TextArea
- TextBox
- TreeGridView
- WebView
- Sample Dialogs
- Related Topics

Guides / Rhino.Python Guides /

Eto Controls in Python

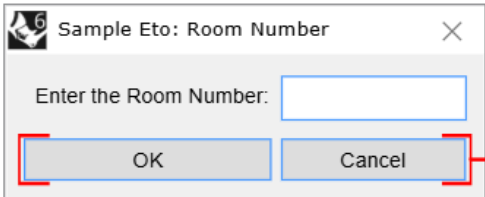
by [Scott Davidson](#) (Last updated: Monday, August 12, 2019)

Eto is an open source cross-platform dialog box framework available in Rhino 6. This guide demonstrates the syntax required to create the most common Eto controls in Rhino.Python. Eto controls include labels, buttons, edit boxes and sliders. In Eto there are [more than 35 different controls](#) that can be created.

For details on creating a complete Eto Dialog in Rhino.Python go to the [Getting Started with Eto Guide](#). The samples in this guide can be added to the controls section of the Basic dialog framework covered in the [Getting Started Guide](#).

Buttons

Buttons are placed on almost every dialog.



Creating a new button is simple. Use the `forms.Button` and specify the `Text` that is shown on the button face. In addition to creating the new button, an action is commonly attached through the `.Click` event. Use the `+=` syntax as shown below to bind the action to button.

```
self.m_button = forms.Button(Text = 'OK')
self.m_button.Click += self.OnButtonClick
```

The bound method, listed later in the class definition is run if the button is clicked.

```
# Close button click handler
def OnOKButtonClick(self, sender, e):
    if self.m_textbox.Text == "":
        self.Close(False)
    else:
        self.Close(True)
```

In this specific case the button is clicked and the bound method `OnOKButtonClick` checks the `Text` of the `m_textbox` to determine if anything has been entered. Then the method closes the dialog, returning either `True` or `False`.

The `Eto.Dialog` class has two special reserved names for buttons, the `DefaultButton` and the

Forms, Modal and Semi-Modal Dialog boxes

Type	Stops Code Execution	Prevents input outside the window
Form	✗	✗
Dialog	✓	✓
Dialog (SemiModal)	✓	✗

Modal vs. Modeless (Forms) Dialog boxes

Modal: Freezes access to the application until the dialog box is closed

Forms: stay open and can be interacted with in combination with application.

For obvious and non-obvious reasons, dialog boxes function differently in ScriptEditor vs. Grasshopper

Forms, Modal and Semi-Modal Dialog boxes

A form dialog box

```
import Eto.Forms as ef

form = ef.Form()
form.Width = 100
form.Height = 100
form.Show()

print("Form launched")
```

A modal dialog box

```
import Eto.Forms as ef

dialog = ef.Dialog()
dialog.Width = 100
dialog.Height = 100
dialog.ShowModal() # <-- Code execution stops here

print("Code returned")
```

A semi-modal dialog box

```
import scriptcontext as sc

from Rhino.UI import RhinoEtoApp, EtoExtensions
import Eto.Forms as ef

parent = RhinoEtoApp.MainWindowForDocument(sc.doc)

dialog = ef.Dialog()
dialog.Width = 100
dialog.Height = 100

# DefaultButton and AbortButton is required for SemiModal
dialog.DefaultButton = ef.Button()
dialog.AbortButton = ef.Button()

EtoExtensions.ShowSemiModal(dialog, sc.doc, parent) # <-- Code execution stops here

print("Code returned")
```

Adding Controls

- [Button](#)
- [DropDown](#)
- [Radio Buttons](#)
- [Check Boxes](#)
- [Text Box](#)
- [Text Area](#)